🧪 Lab: Perform ETL with AWS Glue (S3 → Transform → S3)

🎯 Objective:

Ingest CSV data from Amazon S3, use **Glue Crawler** to catalog it, perform a **transformation using a Glue Job**, and output the data back to S3 in **Parquet** format.

🧱 Lab Architecture Overview

S3 (Raw CSV Data)
 |
 ▼
Glue Crawler → Glue Data Catalog
 |
 ▼
Glue Job (ETL: CSV → Parquet)
 |
 ▼
S3 (Transformed Output)

✅ Step 1: Upload Source Data to S3

1. Go to the **Amazon S3 Console**
2. Create a new bucket (e.g., `glue-etl-lab-source`)
3. Upload a sample CSV file (e.g., `employees.csv`) with contents like:

emp_id,emp_name,dept,salary
101,John Doe,IT,60000
102,Jane Smith,HR,55000
103,Bob White,Finance,65000

🔍 Step 2: Create Glue Crawler

1. Go to the **AWS Glue Console** → **Crawlers** → **Add Crawler**
2. Name: `crawler-employees-csv`
3. Data Source: S3
    - Choose your input bucket/folder (e.g., `s3://glue-etl-lab-source`)
4. IAM Role: Create or choose a role like `AWSGlueServiceRoleDefault`
5. Output:
    - Database: Create new or use existing (e.g., `labdb`)
    - Table name: Will be auto-created from file name
6. Review and finish

▶️ Step 3: Run the Crawler

1. Select the crawler: `crawler-employees-csv`
2. Click **Run Crawler**
3. Once completed:
    - Go to **Data Catalog** → **Tables**
    - Verify that a table was created (e.g., `employees_csv`)
    - Check schema: it should match the CSV headers

🛠️ Step 4: Create a Glue ETL Job

1. Navigate to **Glue** → **Jobs** → **Add Job**
2. Name: `csv-to-parquet-transform`
3. IAM Role: Use the same role as crawler
4. Type: **Spark** (for full ETL)
5. Data Source: Choose `employees_csv` from Data Catalog
6. Transformation Type: **Change schema or format**
   - You can clean, filter, or just convert the format
7. Target:
   - Output to **Amazon S3**
   - New bucket/folder: e.g., `s3://glue-etl-lab-output/employees_parquet/`
   - Format: **Parquet**
8. Save and Run Job

✅ Step 5: Validate Output

1. Go to your output S3 bucket (e.g., `glue-etl-lab-output`)
2. Navigate to the folder `employees_parquet/`
3. Confirm Parquet files are created
4. (Optional) Query using **Amazon Athena**:
   - Create table on top of the output location

- Run SQL: `SELECT * FROM employees_parquet_table;`

🧪 Optional: Add Transformation in Script

Inside the Glue Job Script Editor (PySpark):

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ['JOB_NAME'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

datasource = glueContext.create_dynamic_frame.from_catalog(
    database = "labdb",
    table_name = "employees_csv"
)

# Optional transformation: Filter out employees with salary < 60000
filtered = Filter.apply(frame = datasource, f = lambda x: int(x["salary"]) >= 60000)

glueContext.write_dynamic_frame.from_options(
    frame = filtered,
    connection_type = "s3",
    connection_options = {"path": "s3://glue-etl-lab-output/employees_parquet"},
    format = "parquet"
)

job.commit()
```

🎯 Outcome:

✅ You performed an **end-to-end ETL** using AWS Glue:

- Ingested CSV from S3
- Discovered schema using Glue Crawler
- Transformed & filtered data
- Outputted Parquet to S3
